

Ronin Sydney - u25056507

Ruben van der Vyver - u25007786

Mohammed Lutchka - u25588304

Task 1

1. System Responsibilities & Core Subsystems

The CampusGuard platform provides a centralized emergency-response coordination system supporting the following:

- **Incident Registration & Lifecycle Tracking**
 - a. Registers emergency incidents with descriptive metadata (*type, campus location, severity rating*).
 - b. Tracks state transitions through an explicit lifecycle: Reported → Dispatched → Contained → Resolved
- **Resource Dispatch & Operator Command Execution**
 - a. Dispatches specialized response units (*Campus Security, Medical Responders, Facilities Staff*).
 - b. Enables operators to issue actions (*dispatch unit, secure area, alert, evacuate*) with a built-in cancellation and undo.
- **Access Control & Perimeter Security**
 - a. Executes physical security actions across campus zones, including locking, unlocking, or restricting buildings and rooms.
- **Inter-Component Event Coordination**
 - a. Coordinates collaborating response teams and automated campus services whenever incident conditions or threat levels change.
- **Multi-Channel Notification Broadcasting**
 - a. Broadcasts critical emergency alerts simultaneously across modern digital channels and legacy infrastructure (*e.g., campus paging systems*).
- **High-Level Workflow Orchestration**
 - a. Provides unified, high-level entry points for complex multi-subsystem workflows (*e.g., "Handle Building Emergency"*), coordinating multi-step emergency routines under a single operation.

2. Pattern Mapping

Pattern	Participants
Command	Command: Command (execute(), undo(), describe()). Concrete: DispatchUnitCommand, SecureAreaCommand, IssueAlertCommand, EvacuateCommand. Invoker: OperatorConsole (keeps a history, undoLast() handles cancellation). Receivers: ResponseCenter, AccessControlCentre, AlertService.
Mediator	Mediator: ResponseMediator Concrete: ResponseCenter. Colleagues: ResponseComponent base with SecurityTeam, MedicalTeam, FacilitiesTeam, AccessControlCentre.
Adapter	Target: AlertChannel (send(audience, message, severity)). Adaptee: LegacyPagerSystem (transmitPage(int zoneCode, const char* text, int priority), returns an int status code). Adapter: PagerAlertAdapter. It translates audience → zone code, severity → priority 1-5, std::string → char*, and status code → success/exception. Also there is a modern ConsoleAlertChannel so the adapter is clearly needed.
Facade	Facade: EmergencyResponseFacade with e.g. handleBuildingEmergency(type, building, severity) and standDown(incidentId). Subsystems: IncidentRegistry, ResponseCenter, AccessControlCentre, AlertService
State (extra)	Context: Incident. State: IncidentState Concrete: ReportedState, DispatchedState, ContainedState, ResolvedState, CancelledState.

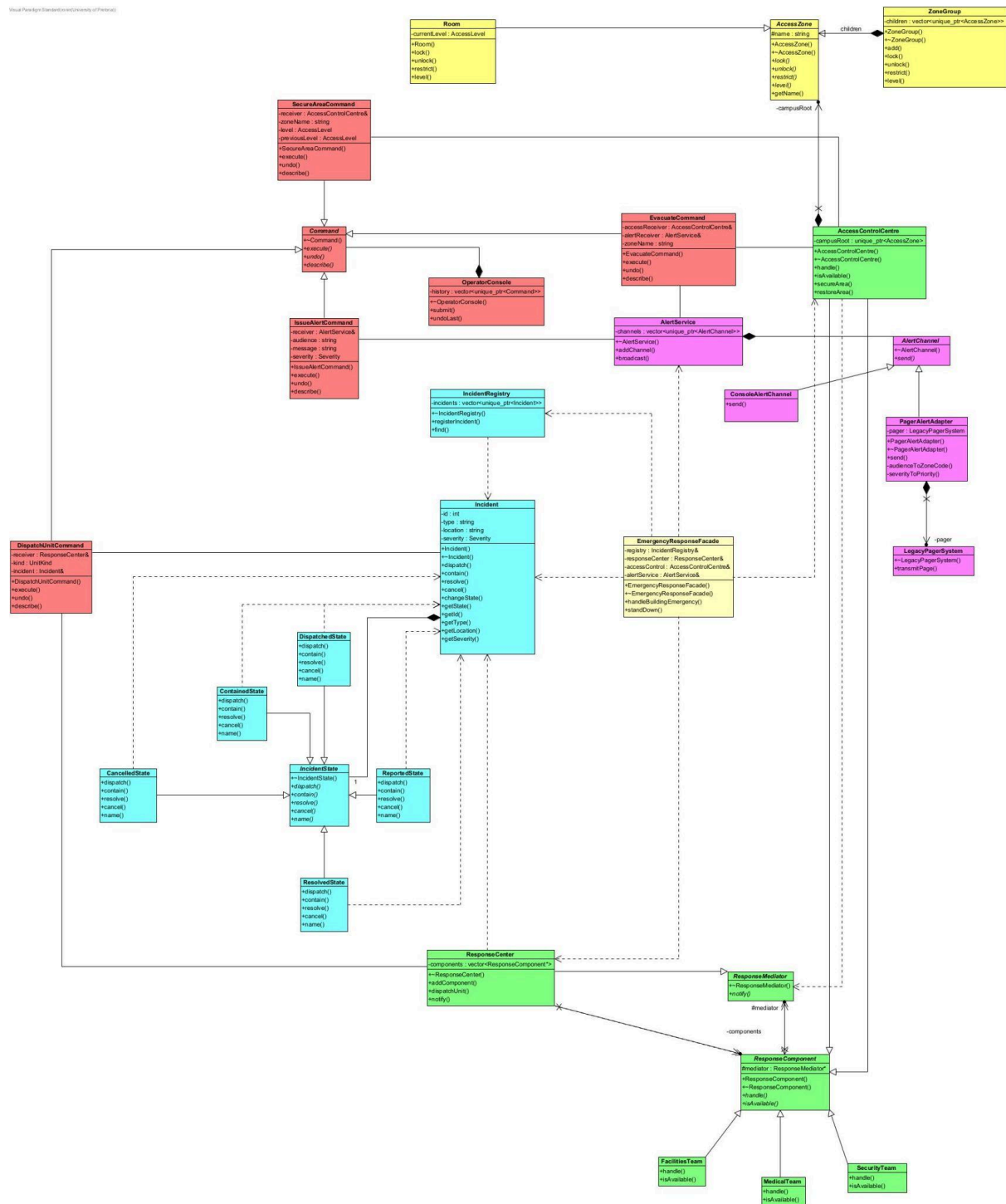
Composite (extra)	Component: AccessZone (lock(), unlock(), restrict()). Leaf: Room. Composite: ZoneGroup (a building or wing).
-------------------	---

3. Justification of the two extra patterns

State (Incident lifecycle). An incident's valid operations depend on its status. You can't resolve one that was never dispatched, and you can't dispatch to a cancelled one. An enum plus switch scatters those rules across the code and is exactly the kind of thing Rule 8 warns against. With State, each status class defines which transitions are legal and throws or reports otherwise. That gives you your required "invalid operation handled sensibly" case for free. Its role in the workflow is that DispatchUnitCommand and the mediator trigger transitions.

Composite (Access zones). Locking "IT Building" should lock every room in it, but locking "Room 2-14" should lock just that room. Without Composite, the command or facade must check whether a target is a building or a room, which is type-checking. With Composite, SecureAreaCommand calls zone->restrict() and never knows or cares which it holds. In the workflow, AccessControlCentre owns the zone tree, and both commands and mediator coordination act on it.

Visual Paradigm Standard (vsn) (University of Pietrasanta)



5. Relationship/Destruction policy

Relationship	Ownership	Destruction
main → subsystems (registry, ResponseCenter, teams, AlertService, facade)	Composition root owns everything, declared in dependency order	Destroyed in reverse order, so receivers outlive commands
IncidentRegistry → Incident	Owns (map<int, unique_ptr<Incident>>)	Registry destructor
Incident → IncidentState	Owns current state, transitions replace it	unique_ptr swap
ResponseCenter → colleagues	Non-owning raw pointers (root owns them)	Not deleted by mediator
Colleague → mediator	Non-owning back-pointer (avoids cycles)	
OperatorConsole → Command history	Owns (vector<unique_ptr<Command>>)	Console Destructor
Command → receivers	Non-owning	Receivers outlive console
ZoneGroup → children	Owns	Recursive via virtual destructor
AlertService → AlertChannels	Owns	Virtual Destructor
PagerAlertAdapter → LegacyPagerSystem	Owns(Composition)	Adapter Destructor
Facade → subsystems	Non-owning references	

Virtual destructors on Command, IncidentState, ResponseComponent, ResponseMediator, AccessZone, and AlertChannel

6. Where the patterns interact

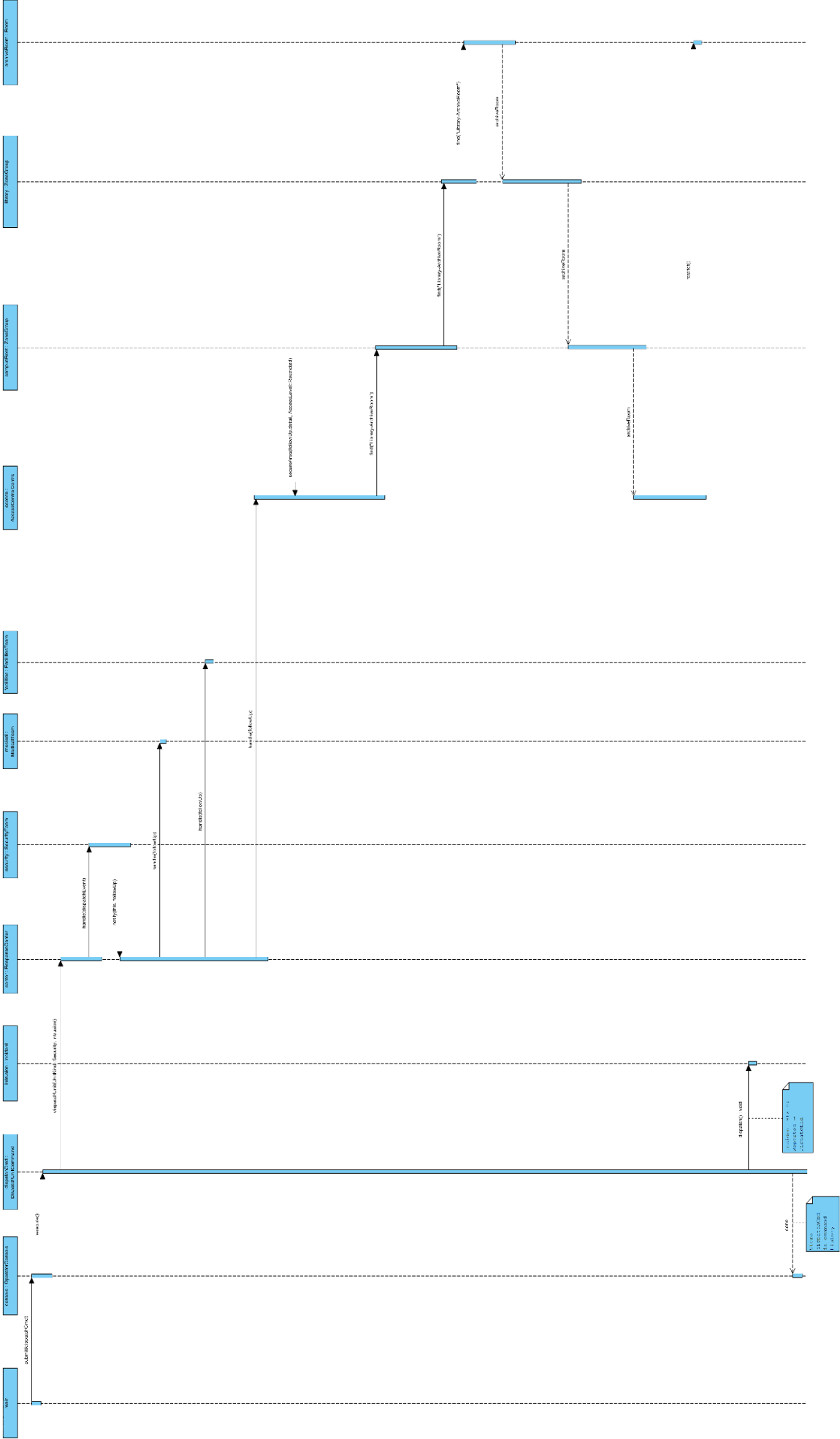
A single flow that covers all six:

1. **Facade** handleBuildingEmergency(...) registers the incident, then builds and submits commands.
2. **Command**: OperatorConsole executes DispatchUnitCommand, and ResponseCenter::dispatchUnit runs as the receiver.
3. **State**: Incident transitions Reported → Dispatched. An invalid attempt (e.g. resolving a Reported incident) is rejected here.
4. **Mediator**: SecurityTeam reports UnitOnScene. It only knows the mediator, and ResponseCenter decides that AccessControlCentre restricts the building and MedicalTeam goes on standby.
5. **Composite**: AccessControlCentre calls restrict() on the building zone, which cascades to all its rooms.
6. **Adapter**: AlertService broadcasts to all channels, including PagerAlertAdapter, which translates into the legacy pager's format.
7. **Command undo**: undoLast() reverses the restriction. That's the cancel action.

Task 4

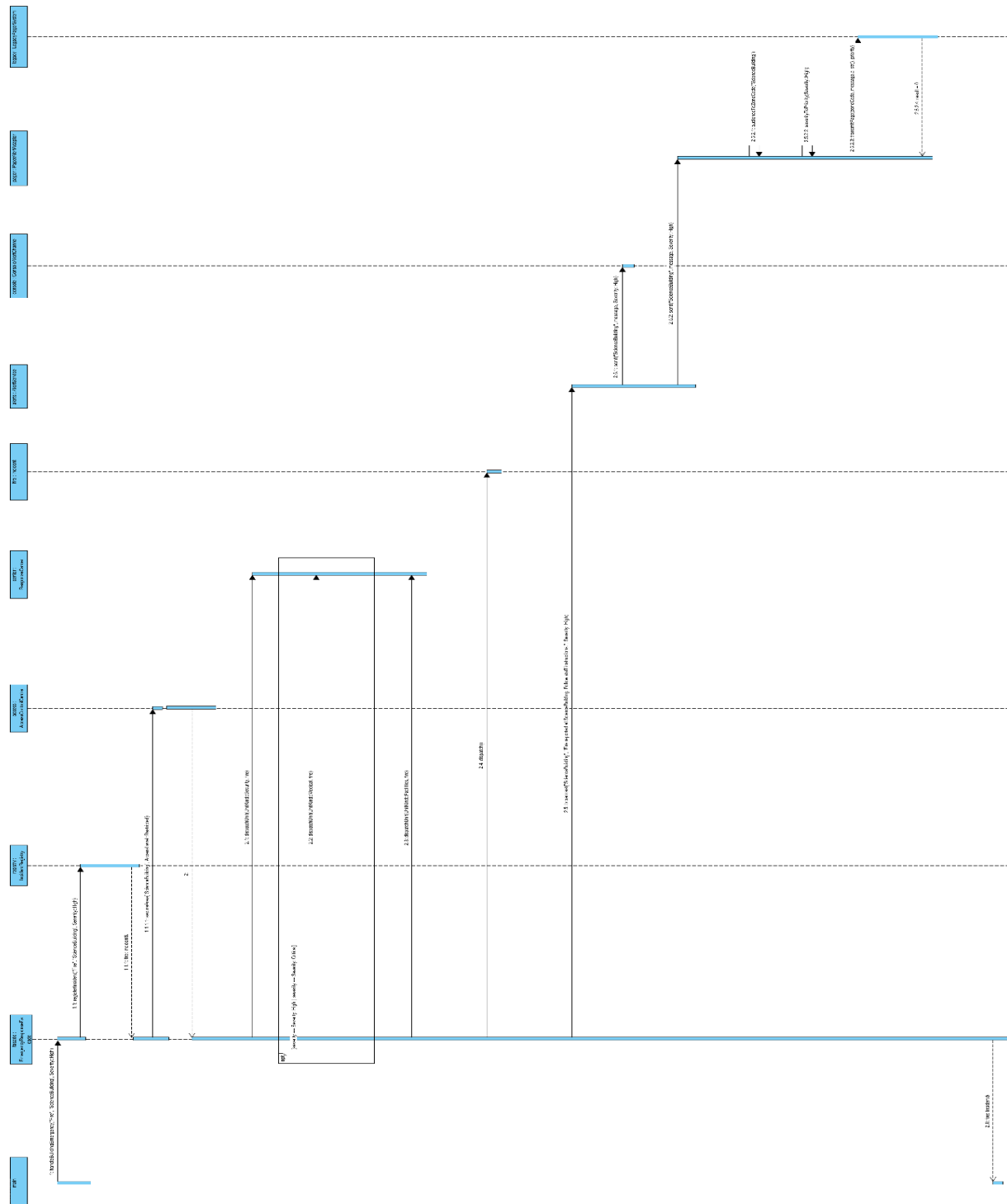
Sequence diagram 1: Library Intrusion – Command and Mediator

This diagram follows a library intrusion request from the operator console through a dispatch command. The command asks the response centre to dispatch a security team. The response centre coordinates the teams, while the security response uses the access system and searches the zone hierarchy for the affected room. It shows Command and Mediator working together during a runtime interaction.



Sequence diagram 2: Science Building Fire – Facade and Adapter

This diagram shows how the emergency response facade handles a reported fire. It registers the incident, restricts building access, dispatches response units, changes the incident's state, and broadcasts an alert. The alert reaches the legacy pager through an adapter, which translates the message into the format the pager system expects.



State diagram: Incident Lifecycle – State Pattern

The state diagram shows how an incident begins as Reported, then moves through Dispatched and Contained to Resolved. An incident can instead be Cancelled while it is Reported or Dispatched. Each labelled transition matches an operation in the incident state implementation, showing how the State pattern controls which changes are allowed.

